



Listas

:::: Introdução a Algoritmos

Trabalhando com Múltiplos Valores

Imagine que precisamos desenvolver um programa que:

- leia 5 notas de um aluno
- calcule a média dessas notas
- exiba o resultado final

Como podemos representar esses dados no programa?

Uma Possível Solução



```
nota1 = float(input("Digite a Nota: "))
nota2 = float(input("Digite a Nota: "))
nota3 = float(input("Digite a Nota: "))
nota4 = float(input("Digite a Nota: "))
nota5 = float(input("Digite a Nota: "))

media = (nota1 + nota2 + nota3 + nota4 + nota5) / 5

print(f"Média: {media}")
```



Agora, é sua vez!

Adapte o programa anterior para:

- ler 5 notas de um aluno
- calcular a média dessas notas
- exibir a média das notas
- exibir apenas aqueles que são maiores que 10

Uma Possível Solução

```
notas = []  
for i in range(5):  
    nota = float(input("Digite a Nota: "))  
    notas.append(nota)  
  
media = (notas[0] + notas[1] + notas[2] + notas[3] + notas[4]) / 5  
  
print(f"Média: {media}")  
  
for i in range(5):  
    if notas[i] > 10:  
        print(notas[i])
```

Limitação do Modelo Atual

E se precisarmos processar 50 notas?

O código funciona, mas **não escala**. Além disso:

- precisamos trabalhar com vários valores
- os valores estão relacionados entre si
- queremos analisar esses dados depois

Estamos trabalhando com vários valores, mas tratando cada um separadamente!

Tipos de Dados

Até agora, trabalhamos com diferentes tipos de dados:

- `int` → números inteiros
- `float` → números reais
- `boolean` → valores lógicos
- `string` → textos

Cada variável armazena um valor por vez!

Um Valor por Variável

Quando usamos variáveis:



```
nota1 = 7.5  
nota2 = 8.0  
nota3 = 6.5
```

Note que:

- cada variável armazena apenas **um valor**
- precisamos criar várias variáveis
- o código cresce rapidamente

Nova Ideia: Estruturas de Dados

Uma estrutura de dados permite:

- armazenar vários valores
- organizar esses dados
- facilitar o processamento

Em vez de várias variáveis, podemos trabalhar com um **conjunto de dados**

Lista

Em Python, uma lista é um **tipo de dado** que:

- armazena múltiplos valores
- mantém uma ordem entre os elementos
- permite acessar e percorrer esses valores



```
notas = [7.5, 8.0, 6.5, 9.0, 8.5]
```

string VS lista

Strings também são **coleções**:



```
texto = "python"
```

Note que uma `string`:

- é uma sequência de caracteres
- é usada para representar texto

Listas são **coleções de dados**:



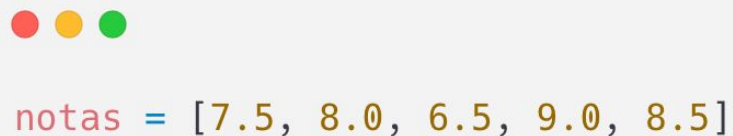
```
valores = [10, 20, 30]
```

Note que uma `lista`:

- possui elementos independentes
- é usada para representar conjuntos de valores

Criando uma Lista

Podemos armazenar **vários valores** em uma única variável:

A code snippet in a light gray box with three colored dots (red, yellow, green) at the top left. The code is `notas = [7.5, 8.0, 6.5, 9.0, 8.5]` where 'notas' is in red, '=' is in blue, and the list values are in yellow.

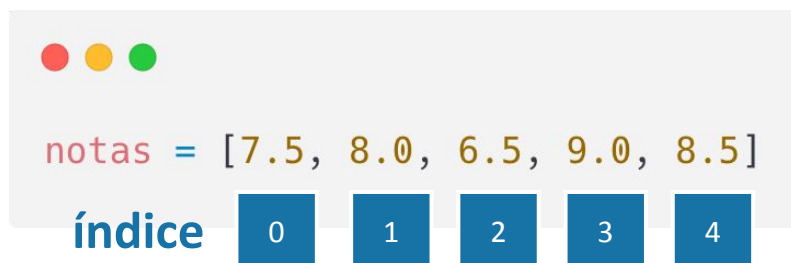
```
notas = [7.5, 8.0, 6.5, 9.0, 8.5]
```

Note que:

- uma variável → vários valores
- os valores são armazenados em sequência
- cada valor ocupa uma posição

Como Pensar em uma Lista

Uma lista funciona como uma **sequência de posições**. Podemos imaginar assim:



The diagram shows a light gray rectangular box. At the top left of the box are three small colored circles: red, yellow, and green. Below these circles is a line of Python code: `notas = [7.5, 8.0, 6.5, 9.0, 8.5]`. The numbers in the list are color-coded: 7.5 is red, 8.0 is yellow, 6.5 is green, 9.0 is red, and 8.5 is yellow. Below the code, the word "índice" is written in blue. To its right are five blue square boxes, each containing a white number: 0, 1, 2, 3, and 4, representing the indices of the list elements.

```
notas = [7.5, 8.0, 6.5, 9.0, 8.5]
```

índice 0 1 2 3 4

Note que:

- cada posição guarda um valor
- acessamos usando o índice
- percorremos elemento por elemento

Lista = dados + ordem + posição

Acessando Elementos da Lista

Podemos acessar os itens de uma lista utilizando os índices:



```
notas = [7.5, 8.0, 6.5]
```

```
print(notas[0])
```

```
print(notas[1])
```

```
print(notas[2])
```

Saída:



```
> 7.5
```

```
> 8.0
```

```
> 6.5
```

Limites da Lista

Cuidados ao acessar posições na lista:

- os índices sempre começam em 0
- o último índice é sempre `len(lista) - 1`
- o acesso à índices inexistentes causa erro

Exemplo: Lendo Vários Valores

No exemplo anterior, criamos a lista manualmente.

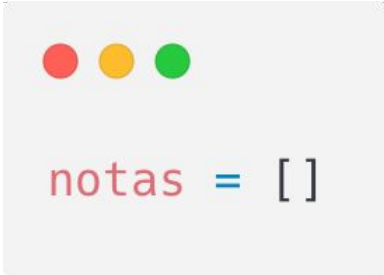
Mas, na prática, precisamos:

- ler valores digitados pelo usuário
- armazenar esses valores
- trabalhar com eles depois

Como podemos construir essa lista durante a execução?

Exemplo: Lendo Vários Valores

Podemos começar com uma **lista vazia**:



```
notas = []
```

Note que:

- ainda não possui elementos
- podemos adicionar valores depois
- a lista cresce ao longo do programa

Exemplo: Lendo Vários Valores

Para **inserir** valores na lista, usamos:



```
notas.append(10.0)  
notas.append(20.0)  
notas.append(30.0)
```

Saída:



```
> [10.0, 20.0, 30.0]
```

Exemplo: Lendo Vários Valores

Podemos ler vários valores e armazenar na lista:

```
● ● ●  
  
notas = []  
  
for i in range(5):  
    nota_do_aluno = float(input("Digite a Nota: "))  
    notas.append(nota_do_aluno)
```

Exemplo: Lendo Vários Valores

Após a inclusão dos valores na lista, teremos:

```
notas = [7.5, 8.0, 6.5, 9.0, 8.5]
```

No código, poderemos:

- analisar os dados
- aplicar regras sobre os valores
- gerar novos resultados

Como podemos percorrer essa lista?

Percorrendo uma Lista

Podemos percorrer os elementos da lista com for:

```
for nota in notas:  
    print(nota)
```

Note que:

- o loop executa uma vez para cada elemento
- a variável `nota` assume cada valor da lista

string VS lista

Antes, percorremos strings:



```
texto = "python"
```

```
for letra in texto:  
    print(letra)
```

Agora, percorremos listas:



```
valores = [10, 20, 30]
```

```
for v in valores:  
    print(v)
```

O comportamento é o mesmo: o `for` percorre coleções!

Exemplo: Contando Notas > 6.0



```
notas = [7.5, 8.0, 6.5, 9.0, 8.5]
```

```
contador = 0
```

```
for nota in notas:  
    if nota > 6.0:  
        contador += 1
```

```
print(contador)
```

Note que:

- percorremos todos os valores
- aplicamos uma condição
- acumulamos o resultado

Exemplo 2: Identificando a Maior Nota



```
notas = [7.5, 8.0, 6.5, 9.0, 8.5]
```

```
maior_nota = notas[0]
```

```
for nota in notas:  
    if nota > maior_nota:  
        maior_nota = nota
```

```
print(maior_nota)
```

Note que:

- começamos com um valor inicial
- comparamos cada elemento
- atualizamos quando necessário

Atenção: Percorrer != Acessar

Temos duas formas de trabalhar com listas:

- percorrendo todos os elementos (**for**)
- acessando posições específicas (**índice**)

Essas duas abordagens serão usadas juntas em vários problemas

Exemplo 2: Identificando a Maior Nota



```
notas = [7.5, 8.0, 6.5, 9.0, 8.5]
```

```
maior_nota = notas[0]
```

```
for i in range(len(notas)):
    if notas[i] > maior_nota:
        maior_nota = notas[i]
```

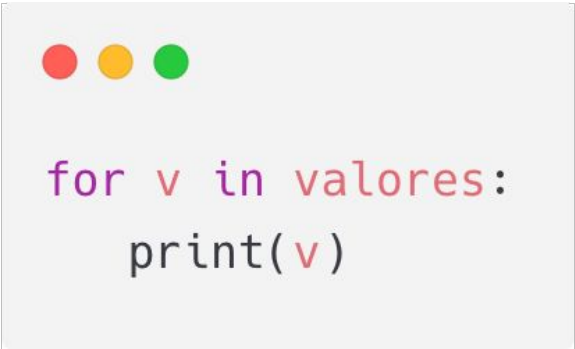
```
print(maior_nota)
```

Também podemos percorrer a lista usando **índices**. Nesse caso:

- `i` representa a posição do elemento
- `notas[i]` acessa o valor naquela posição
- usamos `len(notas)` para controlar o intervalo de posições

For each vs For com Índice

Use for-each quando o foco estiver no **elemento**:



```
for v in valores:  
    print(v)
```

A code snippet in a light gray box with three colored dots (red, yellow, green) at the top left. The code is written in Python and uses a for-each loop to iterate over a list named 'valores', printing each element 'v'.

Use for com índice quando a **posição** também for importante:



```
for i in range(len(valores)):  
    print(i, valores[i])
```

A code snippet in a light gray box with three colored dots (red, yellow, green) at the top left. The code is written in Python and uses a for loop with an index 'i' to iterate over the range of the length of 'valores', printing both the index and the corresponding element 'valores[i]'. The variable 'i' is highlighted in red in the original image.



Agora, é sua vez!

Um sistema precisa armazenar 5 números inteiros digitados por um usuário.

Depois de armazenar todos os valores, o sistema deve exibir esses números na ordem inversa da leitura.

Escreva um programa que:

- leia os 5 números
- armazene os valores em uma lista
- exiba os valores na ordem inversa

Pensando no Problema

Entrada:

- 5 números inteiros

Processamento:

- armazenar os valores em uma lista
- percorrer a lista do final para o início
- acessar os elementos na ordem inversa

Saída:

- valores exibidos na ordem inversa

Casos de Teste

Entrada	Saída Esperada
1, 2, 3, 4, 5	5, 4, 3, 2, 1
10, 20, 30, 40, 50	50, 40, 30, 20, 10
7, 3, 9, 1, 8	8, 1, 9, 3, 7

Implementação

```
valores = []

for i in range(5):
    numero = int(input())
    valores.append(numero)

i = len(valores) - 1

while i >= 0:
    print(valores[i])
    i -= 1
```

Implementação Alternativa

```
valores = []

for i in range(5):
    numero = int(input())
    valores.append(numero)

for i in range(len(valores) - 1, -1, -1):
    print(valores[i])
```

Implementação Alternativa



```
valores = []
```

```
for i in range(5):  
    numero = int(input())  
    valores.append(numero)
```

```
for i in range(len(valores) - 1, -1, -1):  
    print(valores[i])
```

Percorre os índices da lista do último até o primeiro, diminuindo de 1 em 1.

Operações Comuns com Listas

Além de percorrer manualmente a lista, podemos utilizar algumas funções prontas:

```
valores = [12, 7, 25, 3, 18]

print(len(valores))
print(min(valores))
print(max(valores))
print(sum(valores))
```



Dica: Antes de utilizar as funções prontas, tente implementar as suas próprias funções!

O que pode existir em uma lista?

Listas são coleções de **elementos**. Esses elementos podem ser:

- números (`int`, `float`)
- textos (`string`)
- outros tipos de dados

Exemplo:



```
valores = [10, 20, 30]  
nomes = ["Ana", "João", "Maria"]
```

Mistura de Tipos

Em Python, é possível misturar tipos na mesma lista:

```
dados = [10, "Ana", 7.5, True]
```

Mas isso pode gerar problemas:

```
min(dados) # erro
```

- algumas operações deixam de funcionar
- o comportamento pode ficar imprevisível

Boa Prática

Apesar de ser permitido:

- evite misturar tipos na mesma lista
- mantenha um padrão de dados

Listas com dados homogêneos são mais fáceis de entender e usar:



```
valores = [10, 20, 30]  
nomes = ["Ana", "João", "Maria"]
```

Desafio Final: Sequência de Códigos

Um aplicativo de segurança registra os códigos digitados por um usuário durante uma tentativa de acesso. O sistema precisa verificar se houve uma sequência crescente de acessos.

Escreva uma função que receba uma lista de parâmetro e retorne `True` se a lista está em ordem crescente (e `False`, caso contrário).

Exemplo:

- `[3, 7, 10, 15, 20] → True`
- `[3, 7, 6, 15, 20] → False`

Pensando no Problema

Entrada:

- uma lista de códigos numéricos

Processamento:

- percorrer os códigos da lista
- comparar cada código com o código anterior
- verificar se existe alguma quebra na ordem crescente
- retornar False se algum código for menor ou igual ao anterior
- retornar True se nenhuma quebra for encontrada

Saída:

- True ou False

Casos de Teste

Entrada	Saída Esperada
[3, 7, 10, 15, 20]	True
[3, 7, 6, 15, 20]	False
[1, 2, 3, 4, 5]	True
[9, 8, 7, 6, 5]	False
[10, 20, 30, 40, 50]	True

Implementação

```
def esta_crescente(codigos):  
    for i in range(1, len(codigos)):  
        if codigos[i] <= codigos[i - 1]:  
            return False  
  
    return True  
  
codigos = [3, 7, 10, 15, 20]  
resultado = esta_crescente(codigos)  
print(resultado)
```

Desafio Final: Medições de Tempo

Um sistema de análise de desempenho recebe uma lista com os tempos, em segundos, que um atleta levou para completar diferentes tentativas de um mesmo percurso.

Para auxiliar na análise, crie uma função chamada `contar_abaixo_media` que receba uma lista de tempos como parâmetro e retorne a quantidade de tentativas cujo tempo ficou abaixo da média.

Pensando no Problema

Entrada:

- uma lista de tempos

Processamento:

- calcular a soma dos tempos
- calcular a média
- percorrer os valores da lista
- contar quantos estão abaixo da média

Saída:

- quantidade de tempos abaixo da média

Casos de Teste

Entrada	Saída Esperada
[12.5, 11.8, 13.2, 10.9, 12.0]	2
[20, 18, 22, 24, 16]	2
[10, 10, 10, 10, 10]	0
[8, 9, 10, 11, 12]	2

Implementação

```
def calcular_media(lista_valores):  
    soma = 0  
    for valor in lista_valores:  
        soma += valor  
  
    return soma / len(lista_valores)  
  
def contar_abaixo_media(tempo):  
    media = calcular_media(tempo)  
  
    contador = 0  
    for t in tempo:  
        if t < media:  
            contador += 1  
  
    return contador  
  
tempo = [12.5, 11.8, 13.2, 10.9, 12.0]  
resultado = contar_abaixo_media(tempo)  
print(resultado)
```

Desafio Final: Conversor de `string`

Um sistema de análise textual precisa separar uma palavra em partes menores para processar cada caractere individualmente.

Crie uma função chamada `converter_para_lista` que receba uma string como parâmetro e retorne uma lista contendo cada caractere da string em uma posição diferente.

Exemplo:

```
converter_para_lista("python") → ["p", "y", "t", "h", "o", "n"]
```

Pensando no Problema

Entrada:

- uma string

Processamento:

- criar uma lista vazia
- percorrer cada caractere da string
- adicionar cada caractere à lista
- retornar a lista construída

Saída:

- uma lista com um caractere em cada posição

Casos de Teste

Entrada	Saída Esperada
"python"	<code>['p', 'y', 't', 'h', 'o', 'n']</code>
"casa"	<code>['c', 'a', 's', 'a']</code>
"A1b"	<code>['A', '1', 'b']</code>
""	<code>[]</code>

Implementação

```
def converter_para_lista(texto):  
    caracteres = []  
  
    for c in texto:  
        caracteres.append(c)  
  
    return caracteres  
  
resultado = converter_para_lista("python")  
print(resultado)
```

Exercícios de Fixação

1. Escreva uma função que receba uma lista de números e retorne a soma apenas dos valores pares.
2. Escreva uma função que receba uma lista de números e retorne quantos valores são maiores que a média da lista.
3. Escreva uma função que receba uma lista de números e verifique se ela está em ordem crescente.
4. Escreva uma função que receba uma lista de números e verifique se existe algum valor repetido consecutivamente.
5. Escreva uma função que receba uma lista de números e retorne a maior diferença entre dois valores consecutivos.
6. Escreva uma função que receba uma lista de números e verifique se os valores alternam entre par e ímpar.
7. Escreva uma função que receba uma lista de palavras e retorne quantas possuem mais de 5 caracteres.
8. Escreva uma função que receba uma string e retorne uma lista contendo apenas as vogais encontradas.
9. Escreva uma função que receba uma string e retorne uma lista com os caracteres em ordem inversa.
10. Escreva uma função que receba uma lista de números e retorne uma nova lista contendo apenas os valores positivos.
11. Escreva uma função que receba uma lista de números e retorne uma nova lista com cada valor multiplicado por 2.
12. Escreva uma função que receba uma lista de números e retorne True se todos os valores forem positivos, ou False caso contrário.



PUC Minas

© **PUC Minas** • Todos os direitos reservados, de acordo com o art. 184 do Código Penal e com a lei 9.610 de 19 de fevereiro de 1998.

Proibidas a reprodução, a distribuição, a difusão, a execução pública, a locação e quaisquer outras modalidades de utilização sem a devida autorização da Pontifícia Universidade Católica de Minas Gerais.